规范检查工具-pre-commit上手教程

为了防止不符合蓝鲸Python编码规范的代码提交到仓库，每位开发者都应配置pre-commit 代码风格检查

pre-commit是一个代码检查工具，**pre-commit**是由 python 实现的，用于管理和维护多个 pre-commit hooks 的实用框架。它提供了插件式的管理机制，并拥有大量的官方与第三方插件（需要时可自行开发），能够快速实现常见的代码检查任务，如 `eslint` 检查（支持跨语言），`flake8` 检查，`isort` 代码美化等。

Pre-commit的工作原理是在Git的pre-commit钩子中运行配置的检查脚本。当你尝试提交代码时，pre-commit会运行这些脚本，如果任何脚本报告错误，那么提交就会被阻止。

Pre-commit支持多种语言和代码风格检查工具，包括但不限于Python的Flake8，JavaScript的ESLint，CSS的Stylelint等。你可以根据项目的需要，配置不同的检查脚本

```
# 安装 pre-commit
pip install pre-commit

# 初始化 hooks
pre-commit install

执行后，查看 .git/hooks 目录，若创建名为 pre-commit 新文件，则配置成功 。
```

走完以上两步后，可以运行 `pre-commit` 看命令行是否有提示。

```
(py36) ctenetliu@CTENETLIU-MB0 bk-audit-cgs % pre-commit
pyupgrade.....................................(no files to check)Skipped
Check for merge conflicts.....................(no files to check)Skipped
black.........................................(no files to check)Skipped
isort.........................................(no files to check)Skipped
Flake8........................................(no files to check)Skipped
```

pre-commit配置文件

将以下配置文件命名为`.pre-commit-config.yaml`并存放在项目根目录下

```yaml
default_stages: [commit]
repos:
  - repo: https://github.com/asottile/pyupgrade
    rev: v1.21.0
    hooks:
      - id: pyupgrade
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.1.0
    hooks:
    - id: check-merge-conflict
  - repo: https://github.com/psf/black
    rev: 22.8.0
```

```yaml
    hooks:
    - id: black
      language_version: python3.6
      args:
        - "--line-length=120"
  - repo: https://github.com/pycqa/isort
    rev: 5.4.0
    hooks:
      - id: isort
        args:
          - "--profile"
          - "black"
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.1.0
    hooks:
    - id: flake8
      args:
        - "--config"
        - ".flake8"
```

pre-commit的配置文件，它定义了在每次提交前需要运行的钩子（hooks）。下面是对这个配置文件的解读：

- `default_stages: [commit]`：这一行配置了默认的阶段是commit，也就是在每次提交时运行钩子。
- `repos`：这一部分配置了需要使用的代码库和钩子。
    - `repo`：需要使用的代码库的URL。
    - `rev`：需要使用的代码库的版本。
    - `hooks`：需要运行的钩子。
- 对于每个钩子，`id`是钩子的标识符，`args`是传递给钩子的参数。

这个配置文件配置了以下钩子：

- `pyupgrade`：这个钩子来自pyupgrade库，它会自动将Python代码升级到最新版本的语法。
- `check-merge-conflict`：这个钩子来自pre-commit-hooks库，它会检查是否有合并冲突。
- `black`：这个钩子来自black库，它是一个自动格式化Python代码的工具。这个钩子配置了Python版本为3.6，行长度为120个字符。
- `isort`：这个钩子来自isort库，它会自动排序Python的import语句。这个钩子配置了使用black的配置文件。
- `flake8`：这个钩子来自pre-commit-hooks库，它会检查Python代码的风格。这个钩子配置了使用.flake8的配置文件。

这个配置文件的作用是在每次提交时，自动运行这些钩子，以确保代码的质量和风格的一致性。

> 钩子（Hook）在编程中是一种常见的概念，它是一种可以插入到程序的特定位置以改变或增强原有行为的功能。
>
> 在Git中，钩子是一些在特定事件发生时会被执行的脚本。例如，你可以配置一个pre-commit钩子，在每次提交前自动运行一些检查，如果检查失败，则提交会被阻止。

> 在pre-commit工具中，钩子是一些在提交前会被执行的任务，例如代码风格检查，代码格式化，单元测试等。你可以在pre-commit的配置文件中配置需要运行的钩子。
>
> 总的来说，钩子是一种强大的工具，它可以帮助你自动化一些重复的任务，提高开发效率，保证代码质量。

接下来，我们还需要对pre-commit中的各个进行配置和说明。

`pre-commit`是一个用于管理和维护多个预提交钩子的工具。在你的项目中配置`pre-commit`，每次提交代码时，它都会运行一系列的检查。这些检查可以帮助你确保代码的质量和一致性。在上述图片中，有以下几个检查项：

1. `pyupgrade`：这个钩子会自动将你的Python代码升级到最新的语法。例如，它会将`.format`风格的字符串格式化升级为f-string。
2. `Check for merge conflicts`：这个钩子会检查是否有未解决的合并冲突。如果有，它会阻止提交。
3. `black`：Black是一个自动化的代码格式化工具。它会自动将代码格式化为PEP 8风格。这个钩子会在每次提交时运行Black，确保所有的代码都符合PEP 8风格。
4. `isort`：isort是一个Python工具，用于对import进行排序和格式化。这个钩子会在每次提交时运行isort，确保所有的import都被正确地排序和格式化。
5. `Flake8`：Flake8是一个Python工具，用于检查PEP 8风格指南。它可以检查许多PEP 8规定的风格问题，但不包括每一项。这个钩子会在每次提交时运行Flake8，确保所有的代码都符合PEP 8风格。

pre-commit实际使用

接下来，我们在之前的项目中修改部分代码，其中展示一些常见的错误，然后在终端中运行`pre-commit`看能否检测并自动修复。

```
import abc
import hashlib
import json
import logging
import os
import posix
import time

import requests
from requests.exceptions import HTTPError

from bk_plugin.api.constants import KICK_USER_ALLOWED_CODE, SUCCESS_CODE
from bk_plugin.exceptions import APIRequestError

logger = logging.getLogger("bk_plugin_error")
```

注意：在终端中输入 `pre-commit run -a` 来执行代码检查！

```
(py36) [redacted] bk-audit-cgs % pre-commit run -a
pyupgrade.....................................................Passed
Check for merge conflicts.....................................Passed
black.........................................................Passed
isort.........................................................Failed
- hook id: isort
- files were modified by this hook

Fixing /Users/[redacted]/api/start.py

Flake8........................................................Failed
- hook id: flake8
- exit code: 1
```

可以看到，isort和Flake8两个检查项报错了，一般来说直接根据错误信息去溯源修复就可以了，修复好后，再次执行 `pre-commit run -a`，如若代码符合规范，则；

```
bk_plugin/api/start.py:23: [F401] 'posix' imported but unused
1     F401 'posix' imported but unused
1
0
0
0

(py36) [redacted] bk-audit-cgs % pre-commit run -a
pyupgrade.....................................................Passed
Check for merge conflicts.....................................Passed
black.........................................................Passed
isort.........................................................Passed
Flake8........................................................Passed
```

Flake8

Flake8是一个用于Python语言的代码风格检查工具。它可以检查你的Python代码是否符合PEP8编码规范，同时也可以检查代码中的逻辑错误。

Flake8实际上是将PyFlakes，pycodestyle和Ned Batchelder的McCabe script这三个工具集成在一起，因此它既可以检查语法错误，也可以检查代码风格问题，还可以检查代码复杂度。

- PyFlakes：用于检查Python代码中的逻辑错误，比如未定义的变量，未使用的导入等。
- pycodestyle：用于检查Python代码是否符合PEP8编码规范。
- McCabe script：用于检查Python代码的复杂度。

Flake8的优点是使用简单，功能强大，可以高度自定义，因此在Python开发者中非常流行。

.flake8配置文件介绍

```
[flake8]
ignore = F405, W503, E203
exclude =
    *migrations*,
    # python related
    *.pyc,
    .git,
    __pycache__,
    */node_modules/*
    ./static/*
    ./blueking/*
    */blueking/*
    ./blueapps/*
    */blueapps/*
    ./pipeline/*
    ./sites/*
    ./config/default.py
    ./scripts_tools/*
    ./support-files/*
    ./upgrade.py
    ./iam/*

max-line-length=120
max-complexity=26
format=pylint
show_source=False
statistics=True
count=True
```

Flake8是一个Python的代码风格检查工具。下面是对这个配置文件的解读：

- `ignore = F405, W503, E203`：这一行配置了Flake8应该忽略的错误类型。F405是未定义的名称，W503是行尾有操作符，E203是在冒号前有空格。
- `exclude`：这一部分配置了Flake8应该忽略的文件或目录。比如`*migrations*`表示所有名为migrations的目录和文件，`*.pyc`表示所有扩展名为.pyc的文件，`./config/default.py`表示项目根目录下的config/default.py文件。
- `max-line-length=120`：这一行配置了每行代码的最大长度为120个字符。
- `max-complexity=26`：这一行配置了代码的最大圈复杂度为26。圈复杂度是一种代码复杂度的度量，它衡量的是代码中独立路径的数量。
- `format=pylint`：这一行配置了错误信息的格式为pylint格式。
- `show_source=False`：这一行配置了是否显示出错的源代码。False表示不显示。
- `statistics=True`：这一行配置了是否显示统计信息。True表示显示。
- `count=True`：这一行配置了是否显示错误的数量。True表示显示。

这个配置文件的作用是让Flake8按照这些规则来检查代码风格。

常见错误类型

Flake8的错误类型主要分为三类：`F`开头的是来自PyFlakes的错误，`E`和`W`开头的是来自pycodestyle的错误，`C`开头的是来自McCabe的复杂度警告。

以下是一些常见的错误类型：

- `F401`：模块导入了但未使用
- `F403`：'from module import *' used; unable to detect undefined names
- `F405`：name may be undefined, or defined from star imports
- `F841`：局部变量赋值了但未使用
- `E101`：缩进包含混合的制表符和空格
- `E111`：缩进不是四个空格
- `E501`：行太长（超过79个字符）
- `E731`：不要将lambda函数赋值给变量
- `W191`：缩进包含制表符
- `W291`：行尾有空格
- `W293`：行中有空格
- `W601`：.has_key()已废弃，使用'in'代替
- `C901`：函数复杂度过高